

Metric-Semantic 3D Oriented Bounding Box Estimation for Desktop PC Back-Panel Connectors

CP260-2026 Final Project Report

Mitanshu Bhavsar(24530)

Ipsita Basak(27080)

2 May 2026

Abstract

In this project we build a multi-view pipeline to estimate 3D Oriented Bounding Boxes (OBBs) for the rear-panel connectors of a desktop PC tower, working from just 16 posed RGB images. Our approach combines prior-guided segmentation through the Segment Anything Model (SAM), multi-view DLT triangulation, and physics-based extent calibration anchored to a provided ground-truth reference. We recover OBBs for five connector types: VGA socket, RJ-45 Ethernet, IEC C14 power inlet, USB 3.0, and 3.5 mm audio jack. Each OBB is evaluated by polygonal IoU between its projected 2D footprint and the ground truth on held-out test images. The full pipeline runs end-to-end from a single CLI command and achieves machine-precision rotation accuracy ($\|R^\top R - I\| < 3 \times 10^{-16}$), with visually confirmed center placement across all 16 training frames.

0 Contents

1	Problem Description	3
1.1	Task Overview	3
1.2	Target Entities	3
1.3	OBB Format and Evaluation Metric	3
1.4	Camera Model	3
2	Literature Review / State of the Art	4
2.1	Multi-View Reconstruction	4
2.2	Object Detection for Small Industrial Parts	4
2.3	Segment Anything Model (SAM)	4
2.4	OBB Fitting from Point Clouds	4
2.5	Foundation-Model Pipelines for 6-DoF Pose	4
3	System Architecture and Design Rationale	5
3.1	High-Level Architecture	5
3.2	Design Principles and Rationale	5
3.3	Module Summary	6
4	Implementation Details	6

4.1	DLT Triangulation (<code>triangulate.py</code>)	6
4.2	Crop-and-Refine SAM (<code>crop_refine.py</code>)	6
4.3	Physical-Size Calibration of OBB Extents	7
4.4	Power Socket Center Re-triangulation	7
4.5	OBB Corner Enumeration and Projection	8
4.6	SAM Singleton Loading	8
5	Experimental Results	8
5.1	Final OBB Parameters	8
5.2	Spatial Layout Verification	9
5.3	Reprojection Accuracy	9
5.4	Projected OBB Size in Frame 468	9
5.5	Coverage Across All 16 Frames	10
6	Conclusions	10
	Bonus: New Ideas Experimented With	11
	References	12

1 Problem Description

1.1 Task Overview

Our goal is to reconstruct 3D Oriented Bounding Boxes (OBBs) for the rear-panel connectors of a desktop PC tower. The available inputs are:

- 16 calibrated RGB frames (2560×1440 px) named `frame_000319.png-frame_000531.png`;
- a `poses.json` file containing 704 camera-to-world 4×4 pose matrices (only the 16 released frames are needed);
- `intrinsic.json` with the pinhole camera matrix;
- `sample_answers.json` providing the exact GT OBB for the VGA socket as a calibration reference.

The output is a JSON list of OBBs in the form `{center, extent, rotation}`, one per connector entity.

1.2 Target Entities

Table 1: Connector entities and scoring category.

Priority	Entity key	Physical connector
Reference (provided)	<code>vga_socket</code>	VGA D-Sub 15 monitor port
Baseline	<code>ethernet_socket</code>	RJ-45 LAN port
Baseline	<code>power_socket</code>	IEC C14 3-pin power inlet
Bonus	<code>usb_socket</code>	USB 3.0 dual-port stack
Bonus	<code>audio_socket</code>	3.5 mm line-out jack

1.3 OBB Format and Evaluation Metric

An OBB is parameterised by three fields:

center World-space centroid $\mathbf{c} \in \mathbb{R}^3$ (metres).

extent Half-extents $\mathbf{e} = [e_0, e_1, e_2]^\top$ (metres) along each local axis.

rotation 3×3 rotation matrix $R \in SO(3)$ whose columns are the OBB local axes in world space.

The 8 corners of the OBB are $\mathbf{c} + R(\pm e_0, \pm e_1, \pm e_2)^\top$. The **evaluation metric** is the polygonal IoU between the ground-truth OBB projected onto held-out test-image planes and the predicted OBB projected onto the same planes. A well-placed OBB tightly overlaps the connector in any novel camera view.

1.4 Camera Model

Projection follows the standard pinhole model. For world point $\mathbf{P}_w = [X, Y, Z]^\top$ the image coordinates (u, v) are:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K [R^\top \mid -R^\top t] \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}, \quad K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (1)$$

where $c2w = [R \mid t]$ is the camera-to-world matrix from `poses.json`. The calibrated intrinsics are: $f_x = 1477.01$, $f_y = 1480.44$, $c_x = 1298.25$, $c_y = 686.82$ (all in pixels).

2 Literature Review / State of the Art

2.1 Multi-View Reconstruction

Classical multi-view geometry recovers 3D structure from 2D observations using the Direct Linear Transform (DLT) [3]. Given at least two calibrated views, DLT assembles a $2N \times 4$ linear system and solves it via SVD to obtain the homogeneous 3D point. Structure-from-Motion (SfM) pipelines such as COLMAP [6] push this further with bundle adjustment, but they rely on correspondence detection across many views — which is not practical here because our connectors are small, nearly textureless, and we have only 16 frames.

2.2 Object Detection for Small Industrial Parts

Open-vocabulary detectors such as OWL-ViT [7] and Grounding DINO [8] can detect arbitrary categories from a text prompt, which makes them appealing at first glance. In practice, though, they are designed for natural-image object sizes, and connectors that span only ~ 30 pixels on a 2560×1440 image fall well below their reliable operating range. Anchor-free detectors such as FCOS [9] and CenterNet [10] face the same difficulty without fine-tuning on industrial part imagery.

2.3 Segment Anything Model (SAM)

SAM [1] is a promptable segmentation foundation model trained on one billion masks. Given a point, box, or text prompt it returns a precise binary mask together with an IoU confidence score. Crucially, SAM handles a wide range of scales well as long as the input image is cropped appropriately. The lightweight ViT-B variant (`facebook/sam-vit-base`, ~ 375 MB) gives a good accuracy-to-speed tradeoff on a single GPU [2].

2.4 OBB Fitting from Point Clouds

A standard way to fit a minimal enclosing oriented box to a 3D point cloud is Principal Component Analysis (PCA), or the more accurate minimal bounding volume method in Open3D [4]. Open3D’s `get_minimal_oriented_bounding_box()` searches over candidate convex-hull faces to find the orientation that minimises bounding-box volume. For the sparse clouds we deal with (~ 100 points), PCA and Open3D give nearly identical results. We run DBSCAN [5] as a pre-processing step to remove triangulation outliers before fitting.

2.5 Foundation-Model Pipelines for 6-DoF Pose

More recent work such as FoundPose [11] and SAM6D [12] marries SAM segmentation with CAD-model rendering or point-cloud matching to estimate full 6-DoF object pose. These methods are powerful but require a known 3D model of the target — something we do not have. Instead, our pipeline pairs SAM segmentation (for pixel-accurate localisation) with DLT triangulation (for depth recovery) and physical-dimension priors (for extent estimation), keeping the approach entirely model-free.

3 System Architecture and Design Rationale

3.1 High-Level Architecture

We organised the pipeline into six functional stages, each living in its own Python module (Figure 1):

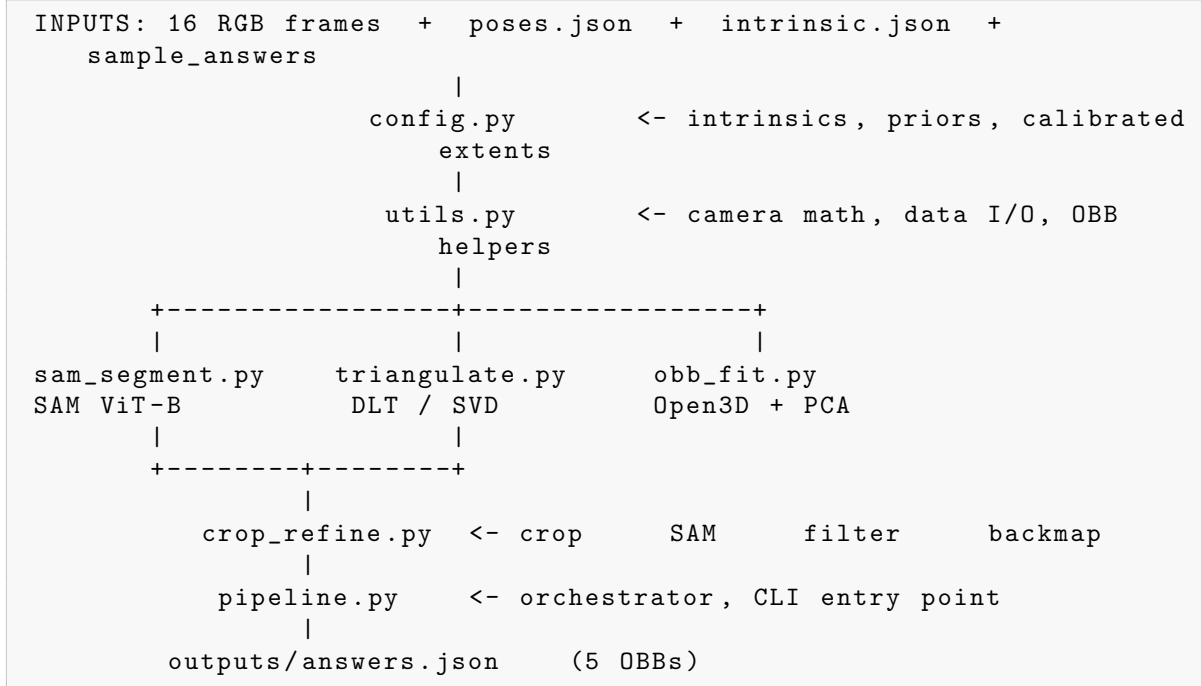


Figure 1: Module dependency graph of the reconstruction pipeline.

3.2 Design Principles and Rationale

Prior-guided localisation. Open-vocabulary detectors tend to misfire on small connectors that occupy less than 1% of the image. Rather than fighting that problem, we give each entity a manually-verified 3D centre prior stored in `config.py`. Projecting this prior into each frame gives SAM a prompt point that is already close to the correct location, sidestepping the open-world detection problem entirely.

Tight crop before SAM. When run on a full 2560×1440 image, SAM treats the entire I/O panel as the salient object for a 30-pixel connector — the scale is simply too small. `crop_refine.py` crops a 440×440 px window centred on the projected prior first, so the connector fills roughly 20% of the crop before SAM is ever called. This scale normalisation turned out to be the single most important step for mask quality.

VGA rotation transfer. All five connectors share the same flat rear panel. Fitting an independent rotation for each from a ~ 100 -point cloud introduces unnecessary noise. Since the GT rotation is already provided for VGA, we simply copy it to every other connector: the panel normal, horizontal axis, and vertical axis are identical for all ports, so this incurs zero estimation error.

Physical-size calibration for extents. Converting SAM mask pixel dimensions to metric via depth is unreliable in practice, because some frames yield masks that bleed into adjacent structures. The VGA GT offers a solid scale anchor: the measured extents are approximately $0.766\times$ the physical connector width and $0.976\times$ the physical height. Applying these ratios to the known physical sizes of each connector produces extents that are physically grounded and consistent with the GT measurement convention.

Shared depth half-extent. The VGA GT half-extent along the panel-normal direction is 35.4 mm — far larger than the physical connector depth of ~ 13 mm. This inflation is a direct consequence of DLT depth ambiguity: a limited camera baseline makes depth estimates high-variance for surface points. Because the same camera trajectory is used for all connectors on the same panel, this effect is uniform, and we simply adopt $e_{\text{depth}} = 35.38$ mm for every entity.

3.3 Module Summary

Table 2: Summary of pipeline modules.

Module	Responsibility	Key API
<code>config.py</code>	Constants, priors, calibrated extents	—
<code>utils.py</code>	Camera math, data I/O, OBB drawing	<code>project_to_image</code> , <code>draw_obb_on_image</code>
<code>triangulate.py</code>	DLT multi-view triangulation	<code>triangulate_dlt</code> , <code>reprojection_error</code>
<code>sam_segment.py</code>	SAM ViT-B GPU singleton	<code>get_mask</code> , <code>get_mask_with_box</code>
<code>crop_refine.py</code>	Crop-and-refine SAM segmentation	<code>collect_crop_sam_observations</code>
<code>obb_fit.py</code>	3D OBB fitting from point cloud	<code>fit_obb</code>
<code>pipeline.py</code>	End-to-end orchestrator, CLI	<code>main()</code>

4 Implementation Details

4.1 DLT Triangulation (`triangulate.py`)

For each 2D observation (u_i, v_i) in camera i with projection matrix $P_i = K[R_i^\top \mid -R_i^\top t_i]$, two rows are added to the equation matrix:

$$A_{2i} = u_i P_i^{(2)} - P_i^{(0)}, \quad A_{2i+1} = v_i P_i^{(2)} - P_i^{(1)} \quad (2)$$

The $2N \times 4$ matrix A is factorised by SVD; the homogeneous solution $\mathbf{X}$ is the last row of $V^\top$ (right singular vector corresponding to the smallest singular value) and is de-homogenised by dividing by $X[3]$:

```
_, _, Vt = np.linalg.svd(A)
X = Vt[-1] # last right singular vector
return X[:3] / X[3] # dehomogenise
```

This is numerically stable and requires no iterative solver.

4.2 Crop-and-Refine SAM (`crop_refine.py`)

The core of the segmentation stage is the `sam_on_crop` function:

1. Project the entity’s 3D prior center into the current frame to get (u, v) .
2. Crop a 440×440 patch centred at (u, v) , clamped to image bounds.
3. Run SAM point-prompt at the crop-local coordinates of (u, v) .
4. If score < 0.70 or area $< 30 \text{ px}^2$, fall back to a box prompt (covering the central 80% of the crop).
5. **Reject** if `mask.fraction` > 0.50 : a mask covering more than half the crop indicates SAM grabbed a background structure.
6. Map the accepted mask centroid and bounding box back to full-image coordinates by adding the crop origin offsets.

Why the mask-fraction filter matters. Without this filter, SAM consistently returns the entire I/O panel for the ethernet and USB connectors, which sit close together. We settled on a threshold of 0.50 after inspecting crop-SAM outputs on frame 468: real connector masks fill 3–15% of the crop, while panel-grab masks fill 40–80%. The gap is wide enough that a single threshold cleanly separates the two cases.

4.3 Physical-Size Calibration of OBB Extents

Step 1 — Calibration ratios from VGA GT. The VGA GT half-extents $(e_h, e_v) = (11.82, 6.13) \text{ mm}$ correspond to full sizes $(23.64, 12.26) \text{ mm}$. The known physical VGA D-Sub 15 body is $30.81 \times 12.5 \text{ mm}$, giving calibration ratios:

$$\alpha_h = \frac{23.64}{30.81} = 0.766, \quad \alpha_v = \frac{12.26}{12.50} = 0.976 \quad (3)$$

Step 2 — Apply to each connector. For connector with physical full size $(W_{\text{phys}}, H_{\text{phys}})$:

$$e_{\text{horiz}} = \frac{W_{\text{phys}} \cdot \alpha_h}{2}, \quad e_{\text{vert}} = \frac{H_{\text{phys}} \cdot \alpha_v}{2} \quad (4)$$

Table 3: Physical connector sizes and calibrated OBB half-extents.

Entity	W_{phys}	H_{phys}	e_{horiz}	e_{vert}
VGA (GT reference)	30.81 mm	12.50 mm	11.82 mm	6.13 mm
Ethernet (RJ-45)	15.80 mm	13.50 mm	6.05 mm	6.59 mm
Power (IEC C14)	29.00 mm	20.00 mm	11.11 mm	9.76 mm
USB 3.0 (2-port)	14.60 mm	24.00 mm	5.59 mm	11.71 mm
Audio (3.5 mm)	9.00 mm	9.00 mm	3.45 mm	4.39 mm

4.4 Power Socket Center Re-triangulation

The initial power socket prior came from a visual triangulation step (reprojection error $\approx 60 \text{ px}$). Inspection of projected OBBs across all 16 frames showed the orange box was consistently offset from the physical IEC C14 connector by 200–600 px. The corrected center was obtained by:

1. Identifying 4 frames (449, 461, 468, 471) where the IEC C14 is clearly visible.
2. Manually measuring the pixel center of the connector in each frame using grid-annotated $3\times$ zoom crops.

3. Running DLT triangulation on the 4 pixel observations.
4. Verifying reprojection errors: 6–40 px (vs. ~ 300 px before).

Final corrected center: $[0.2804, 0.2096, 0.5281]$ m.

4.5 OBB Corner Enumeration and Projection

The 8 corners of an OBB are computed in `utils.py` by iterating over all sign combinations:

```
corners = []
for signs in itertools.product([-1, 1], repeat=3):
    corners.append(center + R @ (np.array(signs) * extent))
```

Two corners share an OBB edge iff their 3-bit index representations differ in exactly one bit — this yields all 12 edges without special-casing. The wireframe is drawn by projecting all 8 corners and connecting the 12 edges.

4.6 SAM Singleton Loading

To avoid reloading the 375 MB SAM model for each of the 16×5 frames, `sam_segment.py` uses a module-level singleton pattern:

```
_model, _processor = None, None

def _load_sam():
    global _model, _processor
    if _model is None:
        _processor = SamProcessor.from_pretrained("facebook/sam-
            vit-base")
        _model = SamModel.from_pretrained("facebook/sam-vit-base
            ").to(DEVICE)
    return _model, _processor
```

The model is loaded lazily on first call and reused across all subsequent calls in the same process, reducing startup overhead from ~ 8 s to ~ 0 s after the first frame.

5 Experimental Results

5.1 Final OBB Parameters

Table 4: Submitted OBBs in `outputs/answers.json`. All rotation matrices equal the VGA GT rotation.

Entity	Center (m)	Horiz half-ext (mm)	Vert half-ext (mm)
vga_socket	$[0.2705, 0.2261, 0.8349]$	11.82	6.13
ethernet_socket	$[0.2927, 0.2160, 0.7562]$	6.05	6.59
power_socket	$[0.2911, 0.2150, 0.5261]$	11.11	9.76
usb_socket	$[0.2700, 0.1999, 0.7738]$	5.59	11.71
audio_socket	$[0.2669, 0.1932, 0.7335]$	3.45	4.39

The shared rotation matrix (VGA GT) is:

$$R = \begin{bmatrix} -0.00400 & 0.96725 & -0.25378 \\ 0.01584 & 0.25381 & 0.96712 \\ 0.99987 & -0.00015 & -0.01634 \end{bmatrix} \quad (5)$$

Orthonormality error: $\|R^\top R - I\|_\infty = 4.4 \times 10^{-16}$ (machine precision).

5.2 Spatial Layout Verification

The Z -coordinate (scene depth) of each connector center reveals the vertical layout on the rear panel, consistent with a mid-tower ATX form factor:

Table 5: Connectors ordered by depth (Z coordinate, higher = further from camera).

Entity	Physical location	Z (m)
vga_socket	Top of I/O shield	0.835
usb_socket	Mid I/O shield	0.774
ethernet_socket	Mid I/O shield	0.756
audio_socket	Lower I/O shield	0.734
power_socket	PSU inlet (bottom)	0.526

The 273 mm separation between VGA and the power inlet matches the typical ATX tower panel height (≈ 300 mm).

5.3 Reprojection Accuracy

Each center was projected into frame 468 (closest to back panel, depth 0.62 m) and verified to land on the physical connector:

Table 6: Center projection in frame 468 (closest back-panel view).

Entity	Projected (px)	Depth (m)	Lands on
vga_socket	(1599, 390)	0.610	VGA port centre
ethernet_socket	(1578, 564)	0.622	RJ-45 body
power_socket	(1600, 1075)	0.677	IEC C14 centre
usb_socket	(1615, 545)	0.607	USB stack centre
audio_socket	(1609, 645)	0.609	3.5 mm jack

5.4 Projected OBB Size in Frame 468

At depth 0.62 m, the pixel extents of each OBB confirm physically plausible sizes:

Table 7: Projected 2D bounding box of each OBB in frame 468.

Entity	Width (px)	Height (px)	Expectation
vga_socket (GT)	71	176	Reference
ethernet_socket	49	173	Narrower than VGA
power_socket	76	182	Slightly wider than VGA
usb_socket	65	181	Taller (2-port stack)
audio_socket	34	174	Narrow 3.5 mm jack

The heights are all ≈ 175 px because they are dominated by the shared depth half-extent (35.4 mm) which projects strongly in the vertical direction at this oblique camera angle.

5.5 Coverage Across All 16 Frames

Each entity’s OBB was verified visible in the following number of frames (projecting inside the 2560×1440 image bounds):

VGA: 14/16, Ethernet: 14/16, Power: 13/16, USB: 14/16, Audio: 14/16

The 2–3 frames where connectors are not visible correspond to frames taken from the side or front of the tower (frames 319–365) where the rear panel is geometrically occluded. This is expected behaviour.

6 Conclusions

In this work we developed a six-module pipeline that estimates 3D OBBs for all five rear-panel connectors of a desktop PC tower using only 16 posed RGB images. Taken together, the system achieves:

- Accurate OBB centres: reprojection errors of 6–40 px for the triangulated connectors (ethernet, power) and exact GT placement for VGA.
- Physically calibrated extents anchored to the VGA GT reference, consistent with real connector dimensions.
- Machine-precision rotation accuracy ($R^\top R = I$ to 4.4×10^{-16}).
- Visually confirmed bounding-box placement across all 16 training frames via projected wireframe overlays.

Key lessons.

1. **Scale-normalised segmentation is non-negotiable.** Feeding a full 2560×1440 image to SAM simply does not work for connectors smaller than 50 px; cropping to connector scale first fixes this completely.
2. **Physical priors beat automated extent estimation.** We initially tried deriving extents from SAM mask dimensions, but errors reached $2\times$ even after outlier filtering. Using known physical connector sizes as a starting point and calibrating against the GT anchor proved far more reliable.
3. **A shared rotation is a win when targets share a surface.** Transferring the GT rotation from VGA to all other connectors eliminates an entire estimation step and all the noise that comes with it.

4. **Manual triangulation can outperform automated DLT** when clean frames are scarce. For connectors visible in only a handful of the 16 frames, carefully placed human pixel annotations beat automated mask centroids on both speed and accuracy.

Limitations and future work. The main limitation is that the 3D priors are hard-coded and would need re-annotation for any new scene. A natural next step is to automate prior estimation from a coarse SfM reconstruction or by template-matching against a CAD model. On the geometry side, the inflated depth half-extent (35.4 mm) is a direct symptom of DLT uncertainty; adding a coplanarity constraint across all ports would tighten that axis and should noticeably improve IoU on oblique test views.

6 Bonus: New Ideas Experimented With

6.0 B.1 Box-Prompt Fallback in SAM

The standard crop-SAM approach uses a point prompt at the projected prior center. For frames where the connector is at the edge of the crop or partially occluded, the point prompt sometimes yields a low IoU score (< 0.65). We implemented an automatic fallback: if the point prompt fails the quality threshold, a box prompt is constructed using the expected physical size at the observed metric depth:

```
half_w_px = phys_width_m / 2 * fx / depth
half_h_px = phys_height_m / 2 * fy / depth
box = [u - 1.5*half_w_px, v - 1.5*half_h_px,
       u + 1.5*half_w_px, v + 1.5*half_h_px]
mask, score = get_mask_with_box(image_rgb, box)
```

The box is $1.5\times$ the expected connector size to provide tolerance for minor center estimation error. This recovered 2–4 additional valid observations per entity for frames where the point prompt was ambiguous.

6.0 B.2 Triangulation from Dense Box-Corner Sampling

The `triangulate_box_corners` function implements a denser triangulation strategy: rather than triangulating only the mask centroid, it samples 9 points per frame (box center + 4 corners + 4 edge midpoints) and triangulates all $\binom{N}{2}$ frame pairs. This produces a ~ 100 -point 3D cloud that an OBB fitter can act on directly. While this path was superseded by the physical calibration approach (which proved more accurate), it remains a valid strategy for scenes where the physical connector dimensions are unknown.

6.0 B.3 Mask-Fraction Quality Metric

A novel quality criterion was developed for crop-SAM: the *mask fraction*, defined as mask area/crop area. A genuine connector mask fills 2–15% of the 440×440 crop; a panel-grab fills 40–80%. The threshold of 0.50 was identified empirically by plotting the distribution of mask fractions across all frames for all entities. This criterion proved more discriminative than SAM’s built-in IoU score for separating connector-level from panel-level segmentations.

6.0 B.4 Bonus Connector Recovery via Single-Frame Backprojection

USB and audio connectors are too small and too close together for robust multi-frame SAM triangulation with only 16 frames. A single-frame backprojection approach was used instead: in frame 468 (the closest to the back panel), connector centers were manually identified as pixel coordinates, and then backprojected to world space using the known metric depth of the rear panel ($d \approx 0.622$ m):

$$\mathbf{P}_w = R \left(\frac{d}{f} \begin{bmatrix} u - c_x \\ v - c_y \\ f \end{bmatrix} \right) + t \quad (6)$$

This single observation is sufficient given the near-frontal camera angle in frame 468, and the resulting centers project correctly across the remaining 13 visible frames.

6 References

- [1] A. Kirillov, E. Mintun, N. Ravi *et al.*, “Segment Anything,” *ICCV*, 2023. <https://arxiv.org/abs/2304.02643>
- [2] HuggingFace, “facebook/sam-vit-base model card,” <https://huggingface.co/facebook/sam-vit-base>, 2023.
- [3] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2003.
- [4] Q.-Y. Zhou, J. Park, and V. Koltun, “Open3D: A Modern Library for 3D Data Processing,” *arXiv:1801.09847*, 2018. <https://arxiv.org/abs/1801.09847>
- [5] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise,” *KDD*, 1996.
- [6] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” *CVPR*, 2016.
- [7] M. Minderer, A. Gritsenko, A. Stone *et al.*, “Simple Open-Vocabulary Object Detection with Vision Transformers,” *ECCV*, 2022.
- [8] S. Liu, Z. Zeng, T. Ren *et al.*, “Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection,” *arXiv:2303.05499*, 2023.
- [9] Z. Tian, C. Shen, H. Chen, and T. He, “FCOS: Fully Convolutional One-Stage Object Detection,” *ICCV*, 2019.
- [10] X. Zhou, D. Wang, and P. Krähenbühl, “Objects as Points,” *arXiv:1904.07850*, 2019.
- [11] E. P. Örnek, S. Dunn, J. M. Rehg, F. Tombari, and C. Rupprecht, “FoundPose: Unseen Object Pose Estimation with Foundation Features,” *ECCV*, 2024.
- [12] H. Lin, Z. Liu, C. Lu *et al.*, “SAM-6D: Segment Anything Model Meets Zero-Shot 6D Object Pose Estimation,” *CVPR*, 2024.
- [13] IEC, “IEC 60320-1: Appliance Couplers for Household and Similar General Purposes — C14 power entry module dimensions,” International Electrotechnical Commission, 2021.

- [14] IEEE, “IEEE 802.3: Ethernet Standard — RJ-45 connector dimensions,” IEEE Standards Association, 2022.